Chapter 4: Monte Carlo Methods

Hoda Zamani

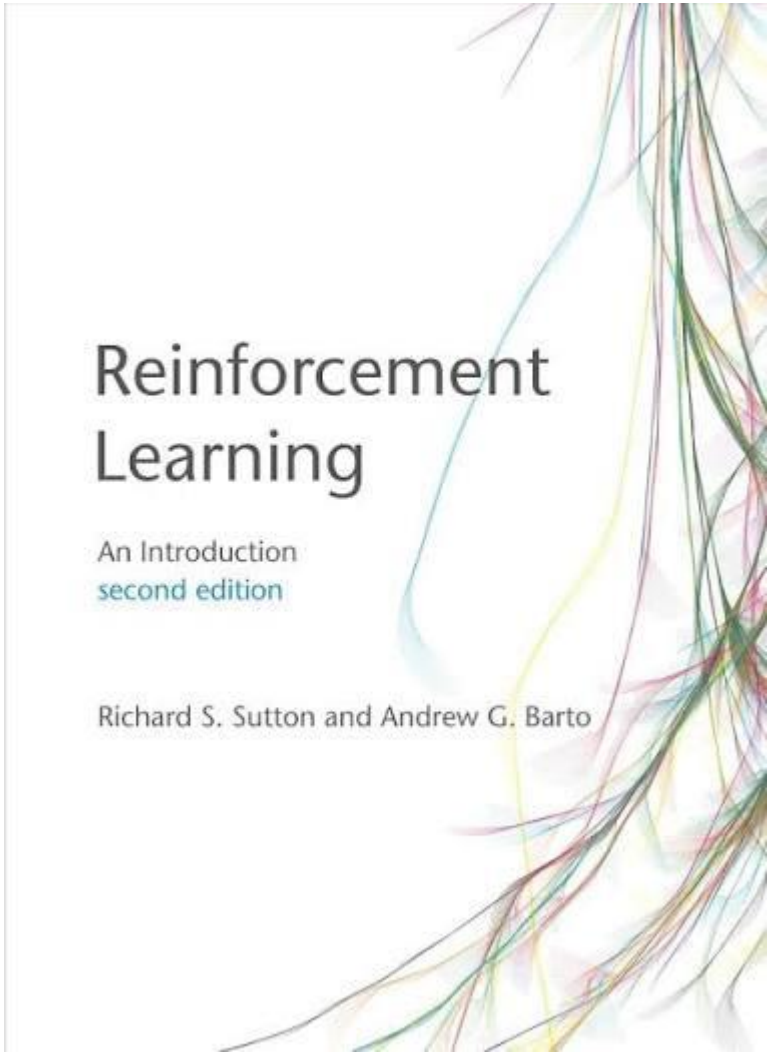
PhD in Artificial Intelligence and Robotics

GitHub: hodazamani.github.io

Hoda Zamani - Google Scholar

Primary Textbook

Reinforcement Learning: An Introduction, Second edition, in progress
Richard S. Sutton and Andrew G. Barto, Download from: SuttonBartoIPRLBook2ndEd.pdf



Recommended References

- Szepesvári, C. (2010). Algorithms for Reinforcement Learning. Morgan & Claypool Publishers.
- Bertsekas, D. P. (2019). Reinforcement Learning and Optimal Control. Athena Scientific.
- Puterman, M. L. (2014). Markov Decision Processes: Discrete Stochastic Dynamic Programming. Wiley.

Additional Course Materials

- Instructor-provided lecture notes, slides, and supplementary reading materials.
- Programming assignments and practical exercises using Python and relevant machine learning libraries.
- Selected research papers and technical articles on reinforcement learning and decision-making.
- Computational demonstrations, datasets, and examples provided throughout the course.

You can find:

 Course announcements & communication
 Slides & weekly schedule
 Textbook & reading materials
 TA
 Course policies

Here: https://hodazamani.github.io/Courses/

Assessment Component	Weight
Weekly Assignments & Programming Exercises	20%
Midterm Examination	30%
Reinforcement Learning Project/Presentation	20%
Final Examination	30%
Total	**100%**

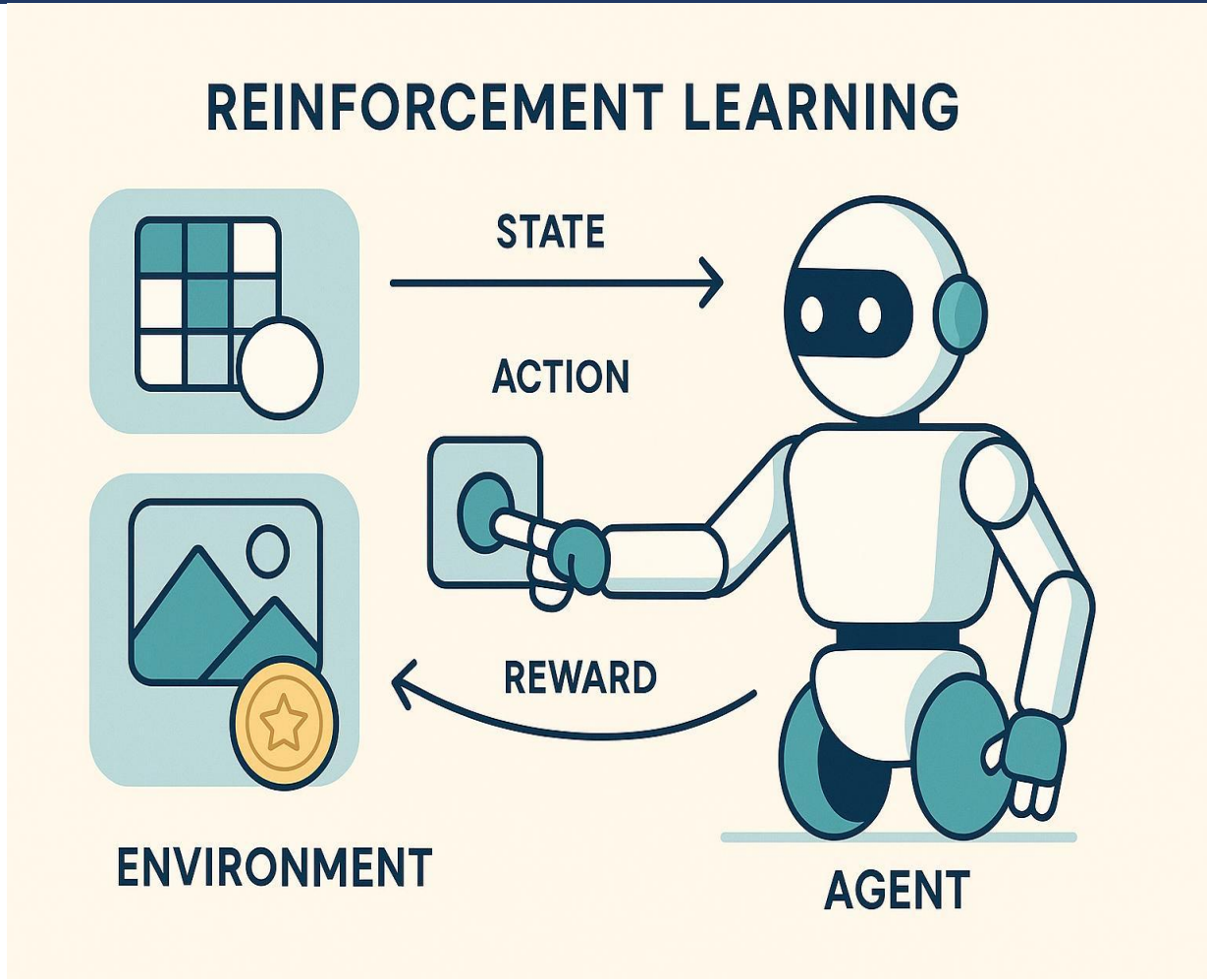
CONTENTS

- **Monte Carlo Prediction**
- **Monte Carlo Estimation of Action Values**
- **Monte Carlo Control**
- **Monte Carlo Control without Exploring Starts**
- **Off-Policy Prediction via Importance Sampling**
- **Incremental Implementation**
- **Off-Policy Monte Carlo Control**
- **Importance Sampling on Truncated Returns**

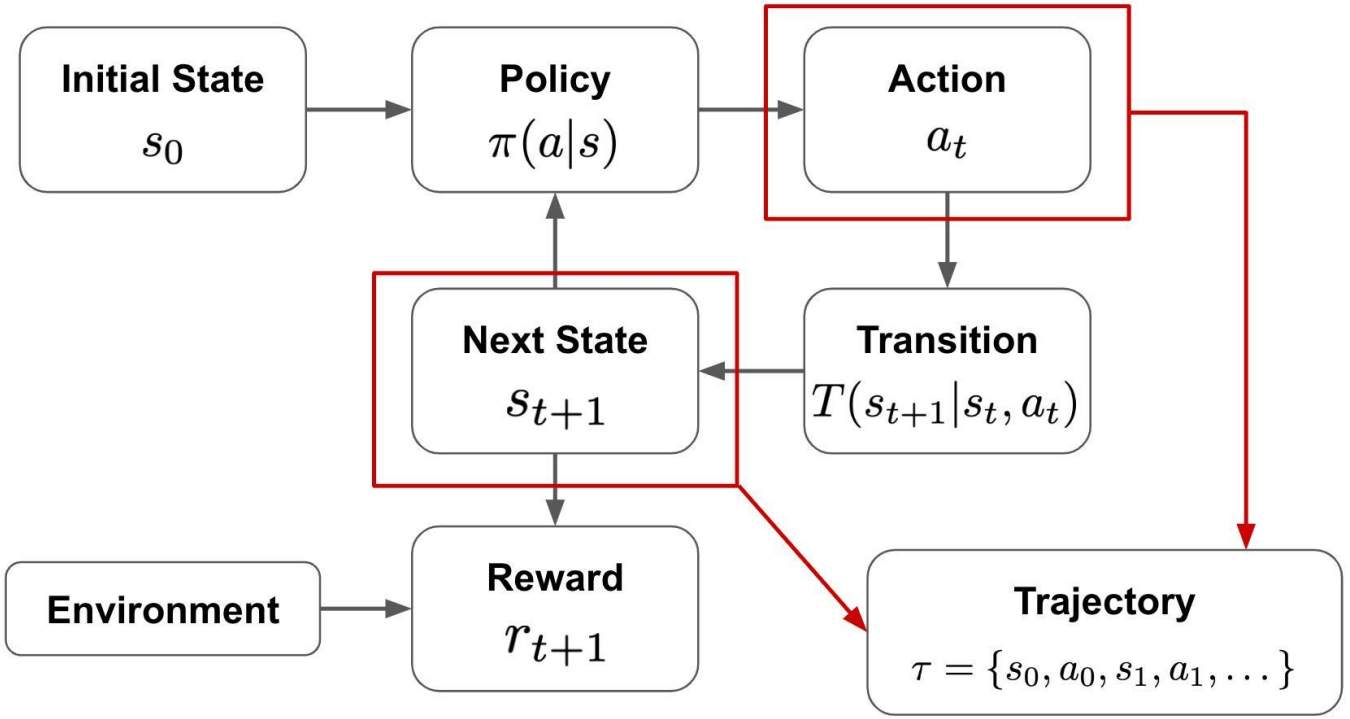
• Monte Carlo (MC) methods are used to estimate the value of states in reinforcement learning. They learn from complete episodes of experience. They estimate state values by averaging the returns observed after visiting a state.

• Unlike Dynamic Programming (DP), MC methods do not require complete knowledge of the environment's transition probabilities.

Instead of calculating every possible outcome mathematically, we play or simulate the task many times and learn from the results.



Dynamic Programming (DP)	Monte Carlo (MC)
Requires a model with transition probabilities	Requires sampled experience
Uses expected outcomes	Uses observed returns
Updates values using other estimated values	Updates values using complete episode returns
Can update without waiting for an episode to finish	Standard MC updates after episode termination



What Is an Episode?

An **episode** is a complete sequence of interactions from a starting point to a terminal state.

For example:

State S_0 → Action A_0 → Reward R_1 → Terminal

- MC methods in this chapter are designed for episodic tasks.
- The episode must eventually terminate.
- Value estimates and policies are updated after the episode finishes.

Return and State Value

The **return** is the total discounted reward received after a particular time step.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$$

Where:

- G_t: return from time t
- R_{t+1}: next reward
- γ: discount factor, between 0 and 1 for the usual discounted setting

The state-value function is the expected return under policy π:

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

Example: If three future rewards are 2, 3, and 5, and $\gamma = 0.9$, then

$$G_t = 2 + 0.9(3) + 0.9^2(5) = 8.75$$

A value function predicts the average long-term return, not just the immediate reward.

Monte Carlo Prediction

Monte Carlo (MC) Prediction is a method in Reinforcement Learning used to **estimate the value of a state under a given policy by averaging the returns observed from episodes**.

The state-value function is:

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

Here:

- $v_\pi(s)$: expected value of state s.
- π: the policy the agent follows.
- G_t: total discounted reward from time t onward.
- $\mathbb{E}_\pi$: expected value when following policy π.

The Monte Carlo estimate

$$\boxed{V(s) = \frac{1}{n}\sum_{i=1}^{n} G_i}$$

This is the **sample-average estimate** used by Monte Carlo Prediction.

It means: average the n observed returns following visits to state s.

- $V(s)$: estimated state value.
- n: number of returns collected.
- G_i: the i-th observed return.

Monte Carlo Prediction

Imagine a robot playing a game. It follows a fixed policy and can visit state A.

Each episode ends with a reward. Suppose the discount factor is $\gamma = 1$, so we simply add the rewards.

Three episodes starting from state A

Episode 1	→	Return = +10
Episode 2	→	Return = −2
Episode 3	→	Return = +4

The robot visits state A in all three episodes. We estimate the value of A by averaging the returns.

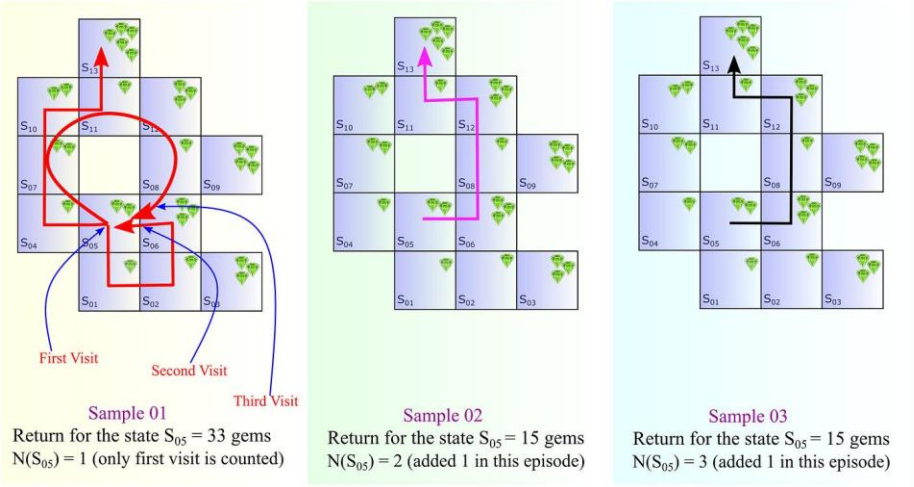
The calculation is:

$$V(A) = \frac{10 + (-2) + 4}{3}$$

$$\boxed{V(A) = 4}$$

Therefore, our estimated value of state A is **4**. This does not mean the robot will always receive 4. It means the observed average return is 4.

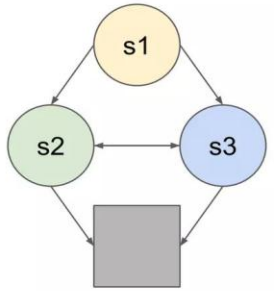
First-Visit and Every-Visit MC



$v(S_{05}) = (33 + 15 + 15) / 3 = 21$ gems

First Visit Monte Carlo State Value Evaluation of State S_{05}

First-visit MC vs. Every-visit MC



- First-visit
  - Average of returns following **first** visits to states
  - Studied widely
  - Primary focus for this chapter
- Every-visit
  - Average returns following **all** visits to states
  - Extended naturally to function approximation (Ch. 9) and eligibility traces (Ch. 12)

Sample Trajectory: s1 → s2 → s3 → s2 → s3 →

First-visit MC

Uses only the return following the first occurrence of a state in each episode.

Every-visit MC

Uses the return following every occurrence of a state in an episode.

Example: Suppose an episode visits state A twice, and the subsequent returns are 8 and 4.

- First-visit MC uses 8 from that episode.
- Every-visit MC uses both 8 and 4.

The difference is how the samples are collected. Both methods converge to the correct state value under the usual assumptions.

First-Visit MC Algorithm

Algorithm workflow

1. Initialize the policy π and value estimates $V(s)$.
2. Create an empty list of returns for every state.
3. Generate a complete episode using π.
4. For every state appearing in the episode, identify its first occurrence.
5. Calculate the return following that first occurrence.
6. Append the return to the state's return list.
7. Recalculate the average to update $V(s)$.

Important: The policy remains fixed during prediction. The objective is to estimate its value, not to improve the policy yet.

First-Visit MC Algorithm

Definition: First-Visit MC is a reinforcement learning method that estimates the value of a state by averaging the returns that follow the **first visit to that state in each episode**.

In simple words, if a state appears multiple times in one episode, we use only its first visit.

Formula:

$$V(s) = \frac{1}{n} \sum_{i=1}^{n} G_i$$

Here, n is the number of episodes in which the state has been visited, and G_i is the return following the first visit in episode i.

Example:

Suppose state A is visited twice in Episode 1.

Episode	First visit return	Second visit return
1	8	4
2	6	2
3	10	5

First-Visit MC uses only the first-visit returns: $8, 6, 10$. $V(A) = \frac{8+6+10}{3} = \boxed{8}$



The second-visit returns $4, 2, 5$ are ignored.

Every-Visit Monte Carlo (MC)

Definition: Every-Visit MC is a reinforcement learning method that estimates the value of a state by averaging the returns that follow **every visit to that state in each episode**.

In simple words, if a state appears multiple times in one episode, we use the return following each occurrence.

Formula:

$$V(s) = \frac{1}{n} \sum_{i=1}^{n} G_i$$

Here, n is the total number of visits to the state, and G_i is the return following each visit.

Example:

Using the same data:

Episode	First visit return	Second visit return
1	8	4
2	6	2
3	10	5

Every-Visit MC uses all six returns: $8, 4, 6, 2, 10, 5$.

$$V(A) = \frac{8+4+6+2+10+5}{6}$$

$$V(A) = \frac{35}{6} \approx \boxed{5.83}$$

All visits are included in the calculation.

First-Visit MC Algorithm

First-Visit MC	Every-Visit MC
Uses only the first visit per episode.	Uses every visit per episode.
A state contributes at most one return per episode.	A state can contribute multiple returns per episode.
Example estimate: 8.	Example estimate: 5.83.

Why Does Monte Carlo Converge?

Monte Carlo (MC) methods converge because **as we collect more episodes, the average of the observed returns approaches the true expected return** under a fixed policy, assuming the standard convergence conditions hold.

Let's understand this step by step.

1. Definition

Convergence means that the estimated state value $V(s)$ approaches the true state value $v_\pi(s)$ as the number of observed returns increases.

The true value is:

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

Monte Carlo estimates this value using the sample average:

$$V(s) = \frac{1}{n}\sum_{i=1}^{n} G_i$$

Here:

- $v_\pi(s)$: true expected return.
- $V(s)$: estimated value.
- G_i: the i-th observed return.
- n: number of returns collected.

Convergence means that $V(s)$ approaches $v_\pi(s)$ as n increases.

Why Does Monte Carlo Converge?

Example: A robot playing a game

Imagine a robot starts in state A. It plays the game repeatedly, following the same policy. Each episode produces a return.



The robot uses these returns to estimate the value of state A.

Step 1: After one episode

The observed return is 10.

$$V(A) = \frac{10}{1} = 10$$

The estimate is 10, but one episode may not represent the typical return very well.

Step 2: After two episodes

The returns are 10 and 6.

$$V(A) = \frac{10+6}{2} = 8$$

The estimate changes from 10 to 8.

Step 3: After three episodes

The returns are $10, 6, 8$.

$$V(A) = \frac{10+6+8}{3} = 8$$

The estimate remains 8.

Step 4: After four episodes

The returns are $10, 6, 8, 4$.

$$V(A) = \frac{10+6+8+4}{4} = 7$$

The estimate changes to 7.

Step 5: After five episodes

The returns are $10, 6, 8, 4, 7$.

$$V(A) = \frac{10+6+8+4+7}{5} = 7$$

These five observations illustrate how an estimate changes as more episodes are collected. They do not prove that the true value is 7; many more episodes may be needed.

Why Does Monte Carlo Converge?

There are three main ideas.

1. Each episode provides a sample.

The return from an episode is one observation of the possible return from state A.

2. Monte Carlo averages the samples.

A single return may be unusually high or low. Averaging multiple returns reduces the influence of individual observations.

3. The Law of Large Numbers explains convergence.

Under the standard assumptions for first-visit MC, the average of independent, identically distributed returns with finite variance approaches their expected value as the number of samples increases.

Mathematically:

$$\frac{1}{n}\sum_{i=1}^{n} G_i \longrightarrow \mathbb{E}[G_i]$$

Therefore:

$$\boxed{V(s) \longrightarrow v_\pi(s)}$$

In simple words, **the more representative experience we collect, the closer the average tends to get to the true expected return.**

4. Does Monte Carlo always give the correct answer?

Not immediately. The estimate can fluctuate, even after many episodes. Convergence is a limiting result: under the appropriate conditions, the estimate approaches the true value as the number of relevant visits grows.

For first-visit MC, these conditions include following a fixed policy and obtaining sufficiently many visits to the state. Every-visit MC also converges under the standard conditions described in reinforcement learning theory.

5. Final summary

Remember these four points:

1. Monte Carlo collects returns from episodes.
2. It estimates state values by averaging those returns.
3. The Law of Large Numbers explains why the average approaches the expected return.
4. As the number of relevant visits grows, the estimated value approaches the true state value under suitable conditions.

Example: Blackjack

- Blackjack is a card game in which the player competes against the dealer.
- The objective is to get a total as close to 21 as possible without exceeding 21.
- The game is an example of an episodic Markov Decision Process (MDP).
- Monte Carlo Prediction estimates the value of game states by averaging returns from completed games.

Concept	Meaning
Agent	The player
Environment	The Blackjack game
State	The player's current total, the dealer's showing card, and whether the player has a usable ace
Action	Hit or stick
Reward	Win: +1, lose: −1, draw: 0
Episode	One complete game
Policy	The rule the player follows when choosing actions
Return	The total discounted reward from a state onward



Example: Blackjack

"In Reinforcement Learning, the player is the agent, and the Blackjack game is the environment. At every decision point, the player observes the current state and chooses an action according to a policy. The game continues until the player wins, loses, or draws. One complete game is called an episode."

In this example, all rewards during the game are zero, and the final reward is $+1$, -1, or 0. The discount factor is $\gamma = 1$, so the return from a state is simply the final game reward.

Example: Blackjack







Imagine an agent learning to play Blackjack. Its goal is to estimate how good a particular state is.

For example, the state might be:

- The player's cards have a total of 18.
- The dealer's visible card is 10.

The agent wants to estimate whether being in this state is good or bad.

Example: Blackjack

Each complete game is called an **episode**.

Suppose the player visits the same state in five games:

Episode	Result	Reward
1	Win	+1
2	Lose	−1
3	Win	+1
4	Draw	0
5	Win	+1

The estimated state value is:

$$V(s) = \frac{1 - 1 + 1 + 0 + 1}{5} = \boxed{0.4}$$

In this example:

- Winning gives a reward of **+1**.
- Losing gives a reward of **−1**.
- A draw gives a reward of **0**.

These rewards are simplified for learning purposes.

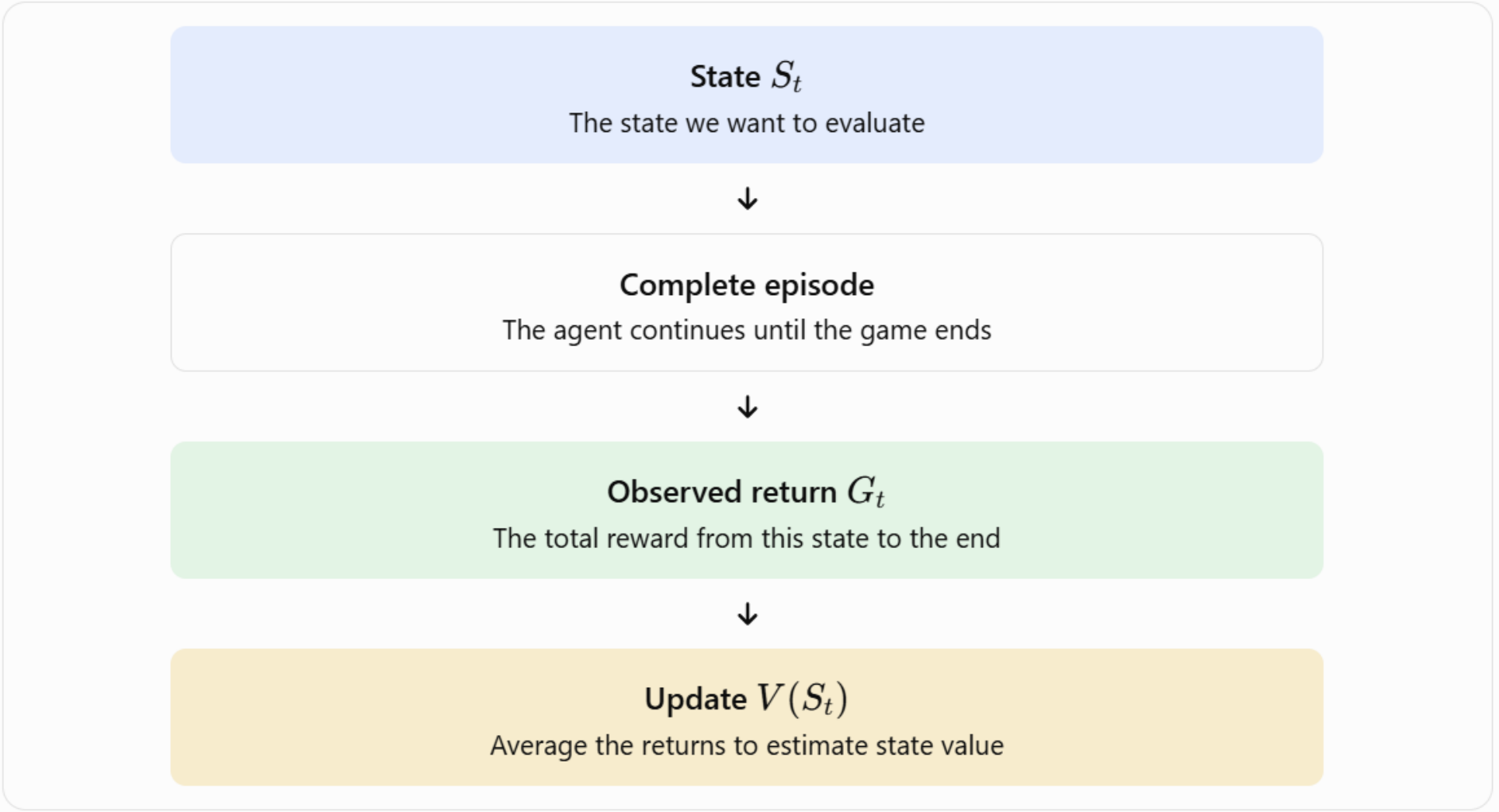
Backup Diagrams in MC

In Reinforcement Learning, a **backup diagram shows how an algorithm uses rewards and future information to estimate the value of a state or action.**

In Monte Carlo (MC), the agent uses the **actual return from a complete episode** to update its estimate of a state value.

In simple words, the agent plays a complete game, observes the final outcome, and uses that experience to improve its estimate.

Monte Carlo backup diagram



The main idea is that the return from the state is backed up to that state to improve its value estimate.

Example using Blackjack

Imagine an agent is playing Blackjack and reaches a state where the player's cards total 18 and the dealer shows a 10.

The agent plays until the game ends.

- Game 1: The agent wins, return $+1$.
- Game 2: The agent loses, return -1.
- Game 3: The agent wins, return $+1$.

The estimated state value is:

$$V(s) = \frac{1 + (-1) + 1}{3} = \frac{1}{3}$$

So, the current estimate of the state's value is approximately 0.33.

The agent uses the actual returns from completed games to improve its estimate. This is the basic idea of an MC backup.

It is called a **backup** because information about future rewards is used to update the estimated value of an earlier state.

MC Does Not Bootstrap

Bootstrapping means updating an estimate using another estimate.

- DP updates a state's value using rewards and estimated values of successor states.
- MC updates a state's value using the return observed from the sampled episode.
- Each state's value estimate can be learned independently, given suitable sampled experience.

For example, MC does not need an estimate of $V(S_{t+1})$ to calculate G_t. It follows the episode and uses its actual sampled rewards.

Advantage: When only a small subset of states matters, we can focus sampling and computation on those states instead of evaluating every state in the environment.

Monte Carlo (MC) does not bootstrap because it uses the actual return from a completed episode instead of using an estimated value of the next state.

- **Bootstrapping** means using an estimate to update another estimate.
- **Does not bootstrap** means using the return observed from the episode as the learning target, rather than another state's estimated value.

Example: Soap Bubble — MC Does Not Bootstrap

Imagine a soap bubble floating in the air. It moves around until it finally bursts.

We can use this as a simple analogy to understand why **Monte Carlo (MC) does not bootstrap.**





Example: Soap Bubble — MC Does Not Bootstrap

1. How does the example work?

Imagine the bubble represents an episode in Reinforcement Learning.

1. **Start:** The bubble begins floating. This represents the agent entering a state.
2. **Continue:** The bubble moves through the air. This represents the agent moving through different states.
3. **Finish:** The bubble bursts. This represents the episode ending.
4. **Calculate:** After the episode finishes, the agent looks at the rewards it received and calculates the return.
5. **Update:** The agent uses that return to update its estimate of the state value.

2. Why does this explain "MC does not bootstrap"?

Suppose an agent receives these rewards during an episode:

$$+1, \quad 0, \quad +2$$

The total return, assuming no discounting, is:

$$G_t = 1 + 0 + 2 = 3$$

MC uses this observed return of 3 as its learning target.

It does **not** need to use an estimated value of the next state to calculate that target.

Monte Carlo Estimation of Action Values

In reinforcement learning, an agent needs to know not only how good a state is, but also which action is best to take in that state.

- **State value** $v_\pi(s)$: the expected return when starting in state s and following policy π.
- **Action value** $q_\pi(s, a)$: the expected return when starting in state s, taking action a, and then following policy π.

The action-value function is

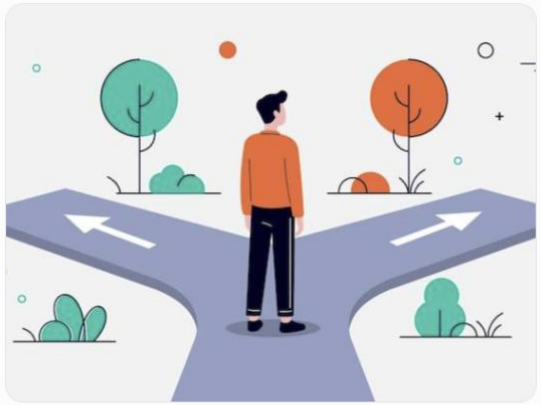
$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

Why are action values important?

When the environment model is unavailable, the agent cannot calculate the consequences of each possible action by looking ahead. Instead, it estimates the values of actions directly from experience.

For example, in blackjack, the agent estimates the expected outcome of both *hit* and *stick* for a particular player hand and dealer card.

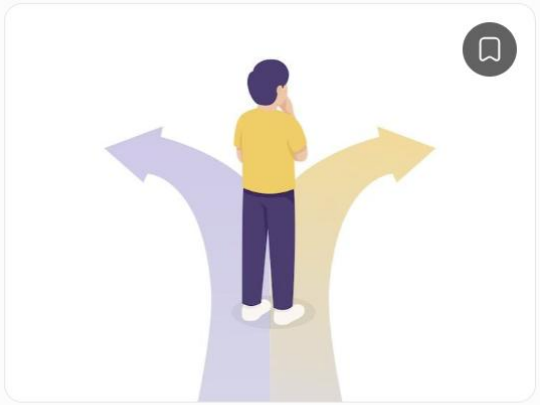
State Values vs. Action Values



State value

$v_\pi(s)$

How good is it to be in this state under policy π?



Action value

$q_\pi(s, a)$

How good is it to take a particular action in this state?

Feature	State value	Action value
Estimates	Value of a state	Value of a state–action pair
Direct action comparison	Not generally sufficient without a model	Yes
Useful for model-free control	Not sufficient by itself	Essential

Main lesson: Action values tell a model-free agent how to compare available actions.

First-Visit and Every-Visit Action-Value MC

The same Monte Carlo averaging methods used for state values can estimate action values.

A state–action pair (s, a) is visited when the agent is in state s and chooses action a.

First-visit MC

Uses the return after the first occurrence of the particular pair (s, a) in each episode.

Every-visit MC

Uses the returns after all occurrences of that pair in each episode.

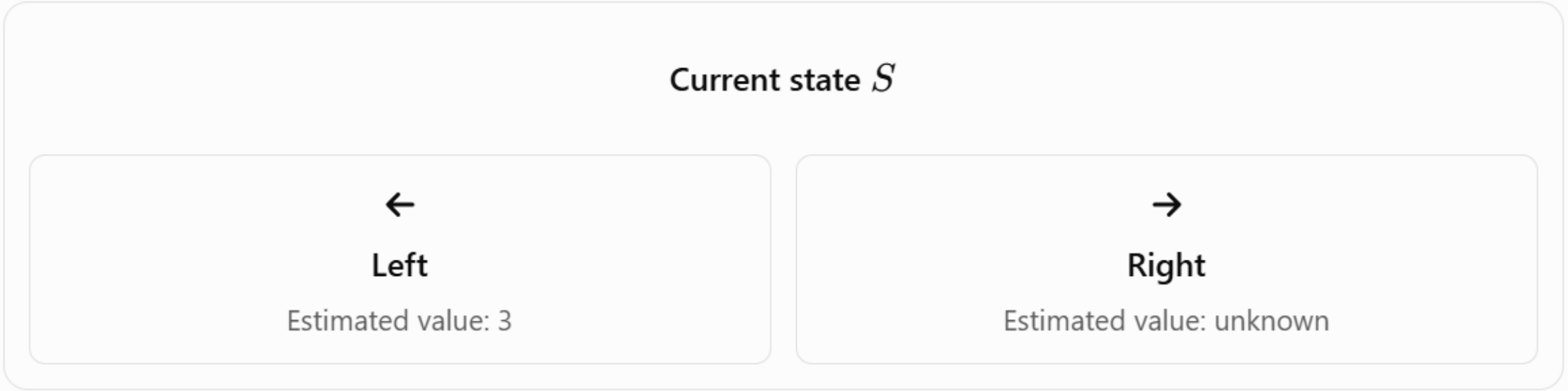
The sample-average estimate is

$$Q(s,a) = \frac{1}{N(s,a)} \sum_{i=1}^{N(s,a)} G_i$$

Here, $N(s, a)$ is the number of returns collected for that state–action pair.

Under the usual episodic assumptions, both approaches converge to $q_\pi(s, a)$ as the number of relevant visits increases without bound.

Suppose a robot can move left or right in a state.



If the agent always chooses left, it may never learn whether right produces a better return.

With a deterministic policy, only the action selected by that policy is normally sampled in each state. The values of unselected actions cannot improve from experience that never includes them.

The exploration problem: The agent must continue trying alternative actions to discover which ones are best.

Exploring Starts

One solution is the **exploring starts** assumption.

Each episode begins from a randomly selected state–action pair, with every possible pair having a nonzero probability of being selected as the starting pair.

The procedure is:

1. Randomly select a starting state S_0.
2. Randomly select a starting action A_0.
3. Generate an episode following the current policy after that initial action.
4. Calculate the returns and update the action-value estimates.
5. Improve the policy using the updated values.

Advantage: Under the appropriate assumptions, every state–action pair can be sampled repeatedly.

Limitation: Real environments often do not let the agent freely choose its initial state and action. Exploring starts is therefore useful for theoretical analysis and simulation, but it cannot always be relied on in practice.

Monte Carlo Control and Generalized Policy Iteration

Monte Carlo control aims to find an optimal policy by repeatedly estimating action values and improving the policy.

This follows the principle of **Generalized Policy Iteration (GPI)**.

Policy evaluation

Estimate $q_\pi(s, a)$ from sampled returns.



Policy improvement

Make better actions more likely using the estimated action values.



Repeat evaluation and improvement until the policy stabilizes.

The two processes influence each other:

- Evaluation improves the accuracy of the value estimates for the current policy.
- Improvement changes the policy based on those estimates.
- The changing policy creates a new value function to learn.

Important: Evaluation and improvement do not need to finish completely before the other begins. Practical GPI can alternate between them after individual episodes.

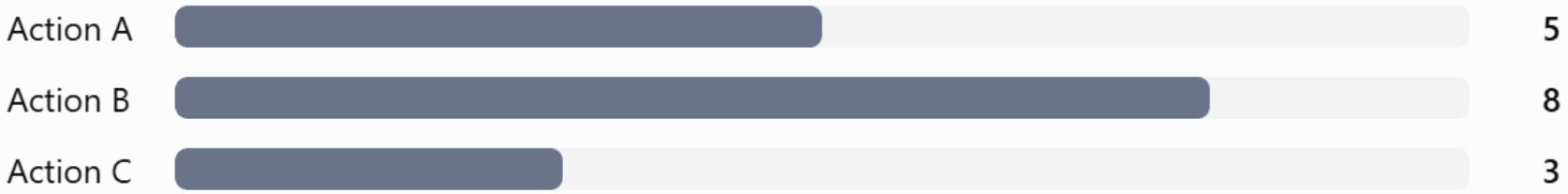
Greedy Policy Improvement

Given an estimated action-value function $Q(s, a)$, a greedy policy selects an action with the highest estimated value.

$$\pi(s) = \arg\max_a Q(s, a)$$

Consider the following example.

Current state: S



Greedy choice: Action B, because its estimated value is 8, the largest of the three.

The policy improvement theorem tells us that a policy made greedy with respect to the exact action values of the current policy is at least as good as the current policy, under the standard assumptions.

Why action values help: The agent can choose the best estimated action without knowing the environment's transition probabilities.

Monte Carlo Exploring Starts (MC ES)

MC ES combines Monte Carlo action-value estimation with greedy policy improvement.

Algorithm workflow

1. Initialize $Q(s,a)$, the policy π, and empty return collections for all state–action pairs.
2. Choose a starting state–action pair so every pair has a nonzero probability of being selected.
3. Generate a complete episode, following the policy after the initial action.
4. Calculate the return following the first occurrence of each state–action pair.
5. Update $Q(s,a)$ by averaging the observed returns.
6. For each state visited, improve the policy by choosing an action that maximizes $Q(s,a)$.
7. Repeat for many episodes.

The key relationship is

$$Q(s,a) \longrightarrow q_\pi(s,a)$$

followed by

$$\pi(s) \longrightarrow \arg\max_a Q(s,a)$$

MC ES is a useful theoretical control algorithm because exploring starts provide coverage of all state–action pairs. However, the classical convergence argument assumes complete policy evaluation; the practical episode-by-episode algorithm does not have the same simple, general convergence proof.

Example: Solving Blackjack with MC ES

Procedure

1. Randomly initialize the player's sum, the dealer's showing card, and whether the player has a usable ace.
2. Randomly select the initial action to support exploring starts.
3. Play the game until it ends.
4. Record the final reward: +1 for a win, −1 for a loss, or 0 for a draw.
5. Use the final return to update the corresponding action-value estimates.
6. Improve the policy by choosing the action with the largest estimated value.



For this task, the return is simply the terminal reward because intermediate rewards are zero and $\gamma = 1$.

Result: After many simulated games, the algorithm learns when hitting or sticking tends to produce better outcomes for each state.

The optimal policy depends on the precise blackjack rules and state representation used.

In real environments, we usually cannot choose every starting state–action pair. The agent must explore while interacting with the environment.

Two approaches address this problem:

On-policy methods	Off-policy methods
Learn about the policy used to generate experience.	Learn about a target policy using data generated by another policy.
Common example: ε-greedy MC control.	Common technique: importance sampling.
Exploration is incorporated into the learning policy.	Behavior and target policies can differ.

The main idea of on-policy control is to make the policy increasingly greedy while ensuring that alternative actions still have opportunities to be selected.

Epsilon-Greedy Policies

An ε-greedy policy usually selects the action with the highest estimated value, but sometimes chooses an alternative action to explore.

- With probability $1 - \varepsilon$, select a greedy action.
- With probability ε, choose an action uniformly at random.

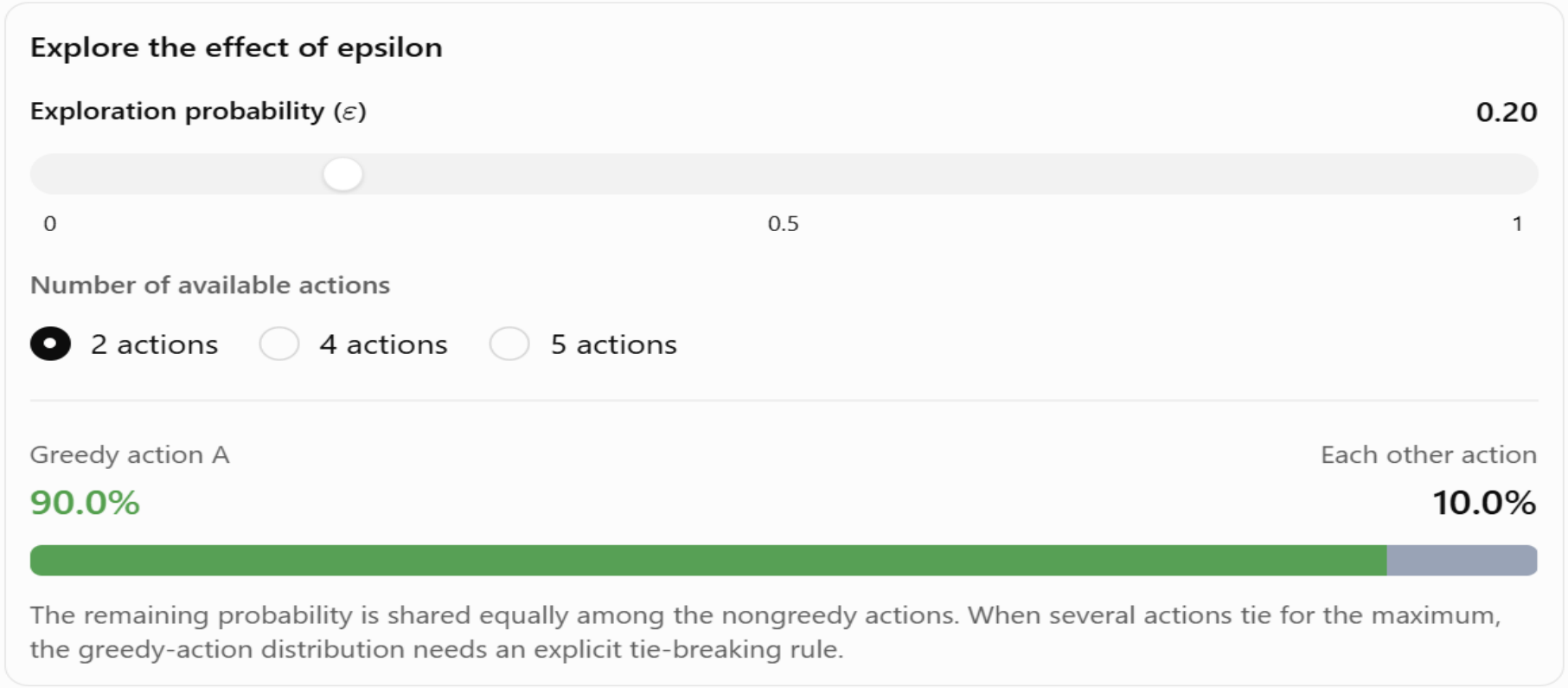
For $m = |A(s)|$ available actions and a unique greedy action, the probabilities are:

$$\pi(a \mid s) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{m}, & a = a^* \\ \frac{\varepsilon}{m}, & a \neq a^* \end{cases}$$

where $a^* = \arg\max_a Q(s, a)$.

Greedy Policy Improvement

Example: Suppose there are two actions, A and B, with A having the highest estimated value. Let $\varepsilon = 0.2$.



Interpretation:

- If $\varepsilon = 0$, the agent always chooses a greedy action.
- If ε increases, exploration becomes more frequent.
- If $\varepsilon = 1$, the agent chooses uniformly at random.
- Keeping $\varepsilon > 0$ ensures every available action has a nonzero probability of selection.

Final summary

The chapter develops a progression from learning values to learning optimal behavior.

Action-value estimation

Estimate $q_\pi(s, a)$ by averaging returns associated with each state–action pair.

Monte Carlo control

Alternate between evaluating actions and improving the policy using GPI.

Exploration

Exploring starts guarantee coverage under their assumptions; ε-greedy policies provide a practical alternative.

Optimal behavior

Repeated evaluation and improvement can lead toward an optimal policy, or the best policy within the specified ε-soft class for on-policy control.

Off-Policy Prediction via Importance Sampling

The main question in this section is:

Can we evaluate one policy using episodes generated by a different policy?

Yes. Monte Carlo methods can do this using **importance sampling**, which adjusts observed returns to account for differences between the policy we want to evaluate and the policy that generated the data.

What Is Off-Policy Learning?



Target policy π

The policy we want to evaluate or improve. We want to estimate $v_\pi(s)$ or $q_\pi(s, a)$.



Behavior policy μ

The policy that actually generates the episodes and supplies the learning data.

In on-policy learning, the same policy generates the data and is evaluated. In off-policy learning, these policies may differ:

$$\mu \neq \pi$$

Example: A robot explores a maze by randomly trying different routes, but we want to evaluate a target policy that follows a particular planned route. Importance sampling lets us use the robot's exploratory episodes to estimate the target policy's expected return.

The Coverage Assumption

Off-policy learning requires **coverage**: every action that the target policy may choose must have a nonzero probability of being chosen by the behavior policy in the same state.

$$\pi(a \mid s) > 0 \implies \mu(a \mid s) > 0$$

Why is this necessary?

Suppose the target policy always chooses right, but the behavior policy never chooses right. The collected data cannot tell us what happens when the agent moves right.

Coverage example

Target policy	Behavior policy
Right: 100%	Right: 50%
Left: 0%	Left: 50%

Coverage is satisfied.

The behavior policy sometimes chooses the action needed by the target policy.

Coverage does not mean the two policies must be identical. The behavior policy can explore more widely, while the target policy can be deterministic.

Importance Sampling

Importance sampling is a statistical technique for estimating an expectation under one probability distribution using samples drawn from another distribution.

In off-policy Monte Carlo learning, we adjust a sampled return using an **importance-sampling ratio**.

For a trajectory from time t until termination at time T:

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k \mid S_k)}{\mu(A_k \mid S_k)}$$

Where:

- $\pi(A_k \mid S_k)$ is the probability of the observed action under the target policy.
- $\mu(A_k \mid S_k)$ is the probability of the same action under the behavior policy.
- The product accounts for all actions in the trajectory from time t through $T-1$.

The weighted return is

$$\boxed{\rho_{t:T-1} G_t}$$

Understanding the ratio:

- Ratio greater than 1: the observed action sequence is relatively more likely under the target policy.
- Ratio between 0 and 1: the sequence is relatively less likely under the target policy.
- Ratio equal to 0: the target policy would never generate that observed sequence.

An important simplification is that the environment's transition probabilities cancel in the ratio. We need the policies' action probabilities, not a complete transition model.

Ordinary Importance Sampling

Ordinary importance sampling calculates the average of the importance-weighted returns.

$$V_{\text{ordinary}}(s) = \frac{1}{N(s)} \sum_{t \in T(s)} \rho_{t:T(t)-1} G_t$$

Here, $N(s)$ is the number of sampled visits to state s, and $T(s)$ is the set of relevant time steps when that state is visited.

Worked example

Suppose we observe two returns for a state:

Episode	Return G	Importance ratio ρ	Weighted return
1	2	2	4
2	4	0.5	2

The ordinary estimate is

$$V_{\text{ordinary}}(s) = \frac{4+2}{2} = \boxed{3}$$

The estimate averages the weighted returns, dividing by the number of observations rather than the sum of the weights.

Main property: Ordinary importance sampling is unbiased under the usual coverage and sampling assumptions, but its variance can be extremely large or even infinite.

Weighted Importance Sampling

Weighted importance sampling normalizes each return by the total importance weight.

$$V_{\text{weighted}}(s) = \frac{\sum_{t \in T(s)} \rho_{t:T(t)-1} G_t}{\sum_{t \in T(s)} \rho_{t:T(t)-1}}$$

If the denominator is zero, the estimate is defined as zero.

Using the same data from Slide 30:

$$V_{\text{weighted}}(s) = \frac{4 + 2}{2 + 0.5} = \boxed{2.4}$$

Compare the two estimates

Ordinary estimate	Weighted estimate
3.0	**2.4**
Divides by 2 observations	Divides by total weight 2.5

These are estimates from a small illustrative sample, not the true value of a particular environment.

Why is weighted importance sampling often preferred?

It typically has much lower variance. With bounded returns, its estimate converges to the target value under standard coverage and sampling conditions, even when the importance ratios have infinite variance.

Ordinary vs. Weighted Importance Sampling

Property	Ordinary	Weighted
Formula	Average of weighted returns	Weighted average of returns
Unbiasedness	Unbiased under standard assumptions	Generally biased with finite samples
Variance	Can be unbounded	Typically much lower
Effect of a large ratio	Can produce extreme estimates	Normalization limits any one observation's normalized weight to at most 1
Practical use	Useful theoretical properties	Often preferred for stability

An important subtlety: Weighted importance sampling is not guaranteed to be less erroneous on every individual sample. Its practical advantage is generally better variance behavior, not that every estimate must be closer to the true value.

Example Blackjack: Off-Policy Prediction

The goal is to estimate the value of **one specific blackjack state** using experience collected from a different policy.

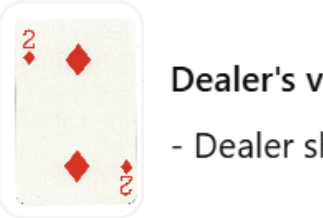
- **Target policy** (π)**:** The policy we want to evaluate.
- **Behavior policy** (μ)**:** The policy used to generate data.
- **Off-policy learning:** Learning about one policy using data generated by another policy.

The key advantage of Monte Carlo methods is that we can estimate the value of a single state directly, without estimating the values of all other states.



Player's hand

- Usable ace
- Player's total: 13



Dealer's visible card

- Dealer shows a deuce (2)

The state we want to evaluate is:

$$s = (\text{player total } 13,\ \text{usable ace},\ \text{dealer shows } 2)$$

A usable ace can count as 11 without making the player's hand exceed 21.

Target Policy vs. Behavior Policy

Target policy π

The policy we want to evaluate

- Stick when the player's sum is 20 or 21.
- Hit otherwise.

Behavior policy μ

The policy that generates the data

- Hit with probability 50%.
- Stick with probability 50%.

Why use two policies?

The behavior policy generates episodes for us, but its decisions do not always match the target policy. Importance sampling corrects for this difference.

The True Value of the State

The book reports the approximate value of this state under the target policy as:

$$\boxed{v_\pi(s) \approx -0.27726}$$

This value was obtained by simulating 100 million episodes using the target policy.

In blackjack, the return is typically:

- $+1$: The player wins.
- 0: The game ends in a draw.
- -1: The player loses.

Therefore, a value of -0.27726 means the expected return is negative: on average, this state leads to an unfavorable outcome under the target policy.

Two Importance-Sampling Methods

Ordinary importance sampling

Each episode's return is multiplied by its importance-sampling ratio.

$$\rho = \frac{\pi(A|S)}{\mu(A|S)}$$

For an episode with multiple decisions, the ratio is the product of the action-probability ratios along the trajectory.

The estimate is:

$$V_{\text{ordinary}} = \frac{1}{n}\sum_{i=1}^{n} \rho_i G_i$$

- Uses the average of weighted returns.
- Can produce very large estimates when weights are large.
- Often has high variance.

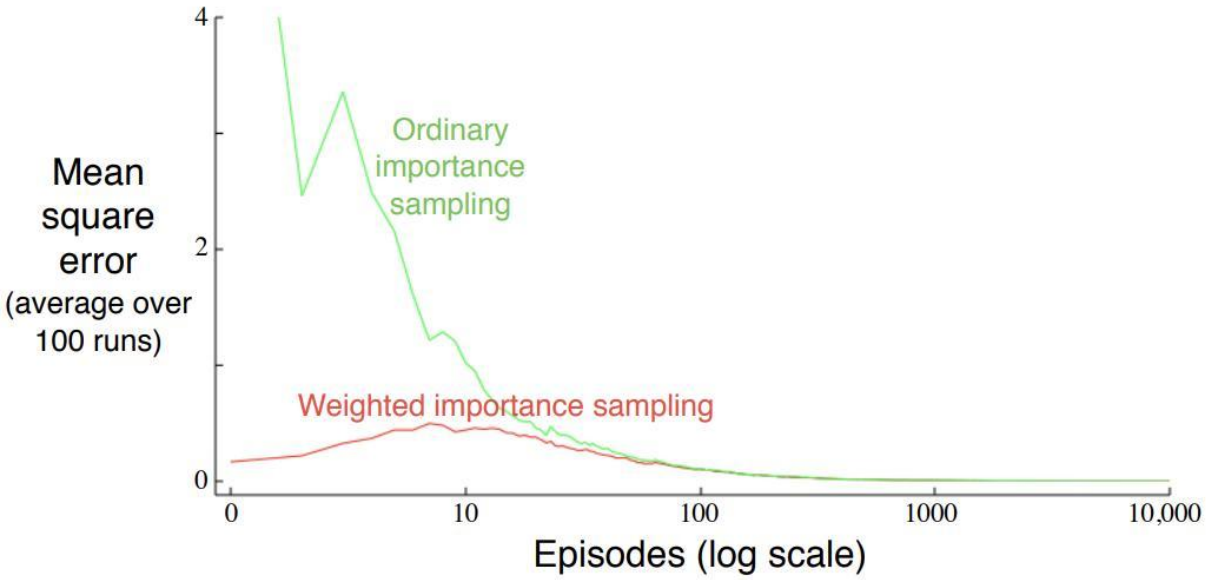
Weighted importance sampling

Instead of dividing by the total number of episodes, this method divides by the sum of the importance-sampling weights.

$$V_{\text{weighted}} = \frac{\sum_{i=1}^{n} \rho_i G_i}{\sum_{i=1}^{n} \rho_i}$$

- Normalizes the weights.
- Usually produces more stable estimates.
- Often has lower mean squared error in practice.

Example Blackjack: Off-Policy Prediction



The graph compares the mean squared error (MSE) of both methods across different numbers of episodes.

- The horizontal axis shows the number of episodes on a logarithmic scale.
- The vertical axis shows mean squared error.
- Each curve summarizes results across 100 independent runs.

Main observation: Weighted importance sampling generally has much lower error than ordinary importance sampling in this example.

The reason is that normalization reduces the influence of unusually large importance-sampling weights.

Example— Why Ordinary Importance Sampling Can Be Unstable

Figure 5.8 introduces a simple one-state Markov decision process (MDP).

- With probability 0.1, the episode ends with reward $+1$.
- With probability 0.9, the process returns to the same state.
- The target policy always chooses the action `back`.
- The behavior policy chooses `back` with probability $1/2$.

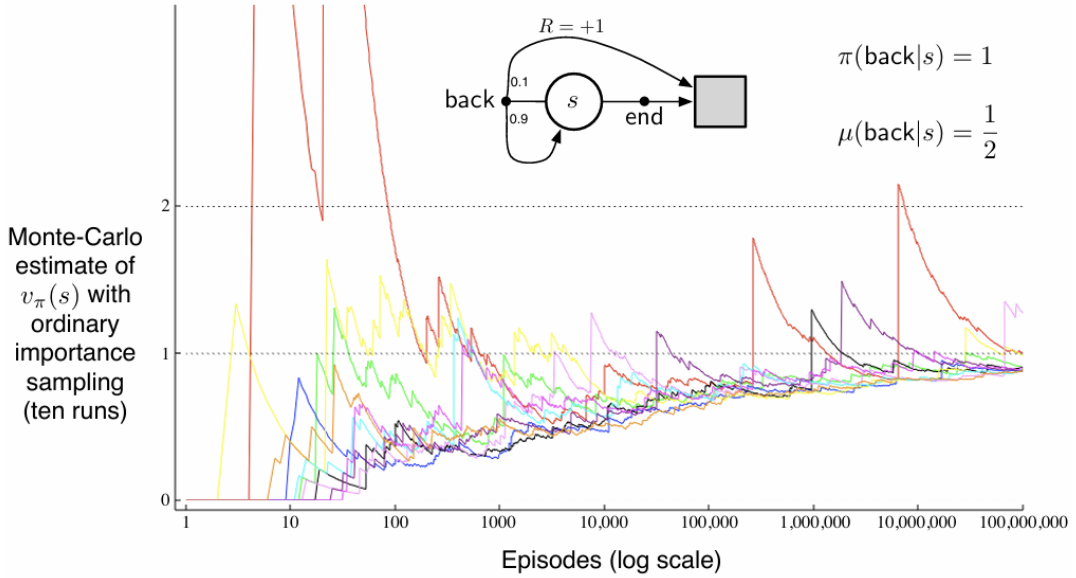
The correct state value is:

$$v_\pi(s) = 1$$

However, ordinary importance sampling can produce extremely unstable estimates.

Why? Rare episodes can receive enormous importance-sampling weights. Even though the expected weighted return is correct, the variance is infinite in this example.

As a result, the estimates do not converge to the correct value in the usual sample-average sense.



Example— Infinite Variance

- Ordinary importance sampling can have infinite variance.
- This can happen when episodes contain loops.
- Very large importance-sampling weights cause unstable estimates.
- A correct expected value does not guarantee reliable estimates in practice.

Understanding the One-State MDP



Slide content

- There is one nonterminal state: s.
- Two actions are available: `end` and `back`.
- `end` terminates immediately with reward 0.
- `back` returns to s with probability 0.9, or terminates with reward $+1$ with probability 0.1.

Target Policy and Behavior Policy

Target policy π

Always chooses back

$$\pi(\text{back} \mid s) = 1$$

$$\pi(\text{end} \mid s) = 0$$

Behavior policy μ

Chooses randomly

$$\mu(\text{back} \mid s) = 0.5$$

$$\mu(\text{end} \mid s) = 0.5$$

Why Is the True State Value Equal to 1?

Under the target policy, the agent always chooses `back`.

- It may return to the state many times.
- Eventually, it terminates with reward $+1$.
- All rewards before termination are zero.

Therefore,

$$G_0 = 1$$

and

$$\boxed{v_\pi(s) = \mathbb{E}_\pi[G_0] = 1}$$

The Importance-Sampling Ratio

The importance-sampling ratio for one action is

$$\rho_t = \frac{\pi(A_t \mid S_t)}{\mu(A_t \mid S_t)}$$

For the action `back`,

$$\rho_t = \frac{1}{0.5} = 2$$

For the action `end`,

$$\rho_t = \frac{0}{0.5} = 0$$

For an episode with k transitions back to s, followed by a final `back` action that terminates the episode, the total ratio is

$$\boxed{\rho = 2^{k+1}}$$

Why Do Large Weights Cause a Problem?

Number of `back` actions	Importance weight
1	2
2	4
3	8
10	1,024
20	1,048,576

For every episode consistent with the target policy, $G_0 = 1$. Therefore, the importance-sampling-scaled return is simply

$$\rho G_0 = \rho$$

Proving That the Variance Is Infinite

For a random variable X,

$$\mathrm{Var}(X) = \mathbb{E}[X^2] - \big(\mathbb{E}[X]\big)^2$$

The mean importance-sampling-scaled return is finite and equals 1. Therefore, we need to examine its second moment:

$$\mathbb{E}[(\rho G_0)^2]$$

For an episode with k loops before termination:

- Probability of the episode: $0.5^{k+1} \times 0.9^k \times 0.1$.
- Importance weight: 2^{k+1}.
- Squared return: 1.

Thus,

$$\begin{aligned}\mathbb{E}[(\rho G_0)^2] &= \sum_{k=0}^{\infty} (0.5)^{k+1}(0.9)^k(0.1)(2^{k+1})^2 \\ &= 0.2\sum_{k=0}^{\infty}(1.8)^k \\ &= \infty\end{aligned}$$

Because the geometric series has a ratio of 1.8, which is greater than one, it diverges.

Therefore,

$$\boxed{\mathrm{Var}(\rho G_0) = \infty}$$

- The expected weighted return is 1.
- The variance of the weighted return is infinite.
- Rare, long episodes can produce extremely large estimates.
- The sample average does not converge to 1 almost surely.

Why Weighted Importance Sampling Works Better Here

Weighted importance sampling calculates

$$V_n = \frac{\sum_{i=1}^{n} \rho_i G_i}{\sum_{i=1}^{n} \rho_i}$$

In this example, every episode consistent with the target policy has

$$G_i = 1$$

Therefore, once at least one such episode is observed,

$$\boxed{V_n = 1}$$

Incremental Implementation of Monte Carlo Methods

- Monte Carlo methods can be implemented incrementally, episode by episode.
- Unlike Chapter 2, which averages rewards, Monte Carlo methods average returns.
- On-policy Monte Carlo methods use the same incremental techniques as sample-average methods.
- Off-policy Monte Carlo methods use two approaches:
  - Ordinary importance sampling
  - Weighted importance sampling

Weighted Importance Sampling

Off-policy learning separates two policies:

- **Behavior Policy** μ : Generates episodes and selects actions.
- **Target Policy** π: The policy being evaluated and improved.

The method uses weighted importance sampling to estimate action values.

The target policy is updated greedily:

$$\pi(s) = \arg\max_a Q(s, a)$$

The behavior policy must provide sufficient exploration by assigning nonzero probability to actions that the target policy may select.

Weighted Importance Sampling Algorithm

1. Initialize action-value estimates (Q(s,a)) and cumulative weights (C(s,a)).
2. Initialize the target policy as greedy with respect to (Q).
3. Generate an episode using the behavior policy μ
4. Process the episode backward, from the final time step to the first.
5. Update the return:

Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:
$Q(s,a) \leftarrow$ arbitrary
$C(s,a) \leftarrow 0$
$\pi(s) \leftarrow$ a deterministic policy that is greedy with respect to Q

Repeat forever:
Generate an episode using any soft policy μ:
$S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T, S_T$
$G \leftarrow 0$
$W \leftarrow 1$
For $t = T-1, T-2, \ldots$ downto 0:
$G \leftarrow \gamma G + R_{t+1}$
$C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} [G - Q(S_t, A_t)]$
$\pi(S_t) \leftarrow \operatorname{argmax}_a Q(S_t, a)$ (with ties broken arbitrarily)
$W \leftarrow W \frac{1}{\mu(A_t|S_t)}$
If $W = 0$ then ExitForLoop

Importance Sampling on Truncated Returns

Conventional importance sampling uses the full importance-sampling ratio for an entire return.

However, when the discount factor γ is small, later rewards contribute less to the return. Using the full ratio may introduce unnecessary variance.

Proposed solution: Use truncated importance-sampling ratios.

A flat partial return is defined as:

$$\bar{G}_t^h = \sum_{k=t+1}^{h} R_k$$

where (h) is the selected horizon.

The full discounted return can be represented as a weighted combination of flat partial returns.

Question

Consider a reinforcement learning problem in which an agent wants to estimate the value of a state s under a target policy π, but the episodes are generated using a behavior policy b.

Suppose the agent observes the following three episodes, all starting from state s:

Episode	Return G	Importance-sampling ratio ρ
1	5	2
2	10	0.5
3	4	1.5

Answer the following questions:

(a) Calculate the ordinary importance-sampling estimate of $V^{\pi}(s)$.

(b) Calculate the weighted importance-sampling estimate of $V^{\pi}(s)$.

(c) Explain the difference between the two estimates and identify which estimator generally has lower variance.

(d) Explain why importance sampling is necessary when the target policy differs from the behavior policy.

Question

(a) Ordinary importance-sampling estimate

The ordinary importance-sampling estimator is

$$V_{\text{ordinary}}(s) = \frac{1}{n} \sum_{i=1}^{n} \rho_i G_i$$

Given $n = 3$, we substitute the values:

$$V_{\text{ordinary}}(s) = \frac{1}{3} \left[(2)(5) + (0.5)(10) + (1.5)(4) \right]$$

Calculate each product:

$$(2)(5) = 10$$

$$(0.5)(10) = 5$$

$$(1.5)(4) = 6$$

Therefore,

$$V_{\text{ordinary}}(s) = \frac{10 + 5 + 6}{3} = \frac{21}{3}$$

Final answer:

$$\boxed{V_{\text{ordinary}}(s) = 7}$$

(b) Weighted importance-sampling estimate

The weighted importance-sampling estimator is

$$V_{\text{weighted}}(s) = \frac{\sum_{i=1}^{n} \rho_i G_i}{\sum_{i=1}^{n} \rho_i}$$

Substitute the given values:

$$V_{\text{weighted}}(s) = \frac{(2)(5) + (0.5)(10) + (1.5)(4)}{2 + 0.5 + 1.5}$$

Simplify the numerator:

$$10 + 5 + 6 = 21$$

Simplify the denominator:

$$2 + 0.5 + 1.5 = 4$$

Thus,

$$V_{\text{weighted}}(s) = \frac{21}{4}$$

Final answer:

$$\boxed{V_{\text{weighted}}(s) = 5.25}$$

(c) Difference between the estimators

The ordinary importance-sampling estimate is 7, while the weighted importance-sampling estimate is 5.25.

The difference is due to how the importance weights are handled.

- **Ordinary importance sampling** divides the weighted sum of returns by the number of episodes.
- **Weighted importance sampling** divides the weighted sum of returns by the sum of the importance weights.

Weighted importance sampling generally has lower variance and is often more stable when importance ratios vary substantially. However, it is generally biased for finite samples, whereas ordinary importance sampling is unbiased under standard assumptions.

(d) Why is importance sampling necessary?

The behavior policy b generates the episodes, but the goal is to estimate the value under the target policy π.

These policies may choose actions with different probabilities. Consequently, directly averaging returns generated by b generally estimates $V^b(s)$, rather than $V^\pi(s)$.

Importance sampling corrects for this difference using the ratio

$$\boxed{\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k \mid S_k)}{b(A_k \mid S_k)}}$$

This ratio adjusts each observed return according to how likely the sampled actions are under the target policy relative to the behavior policy.

Any questions?